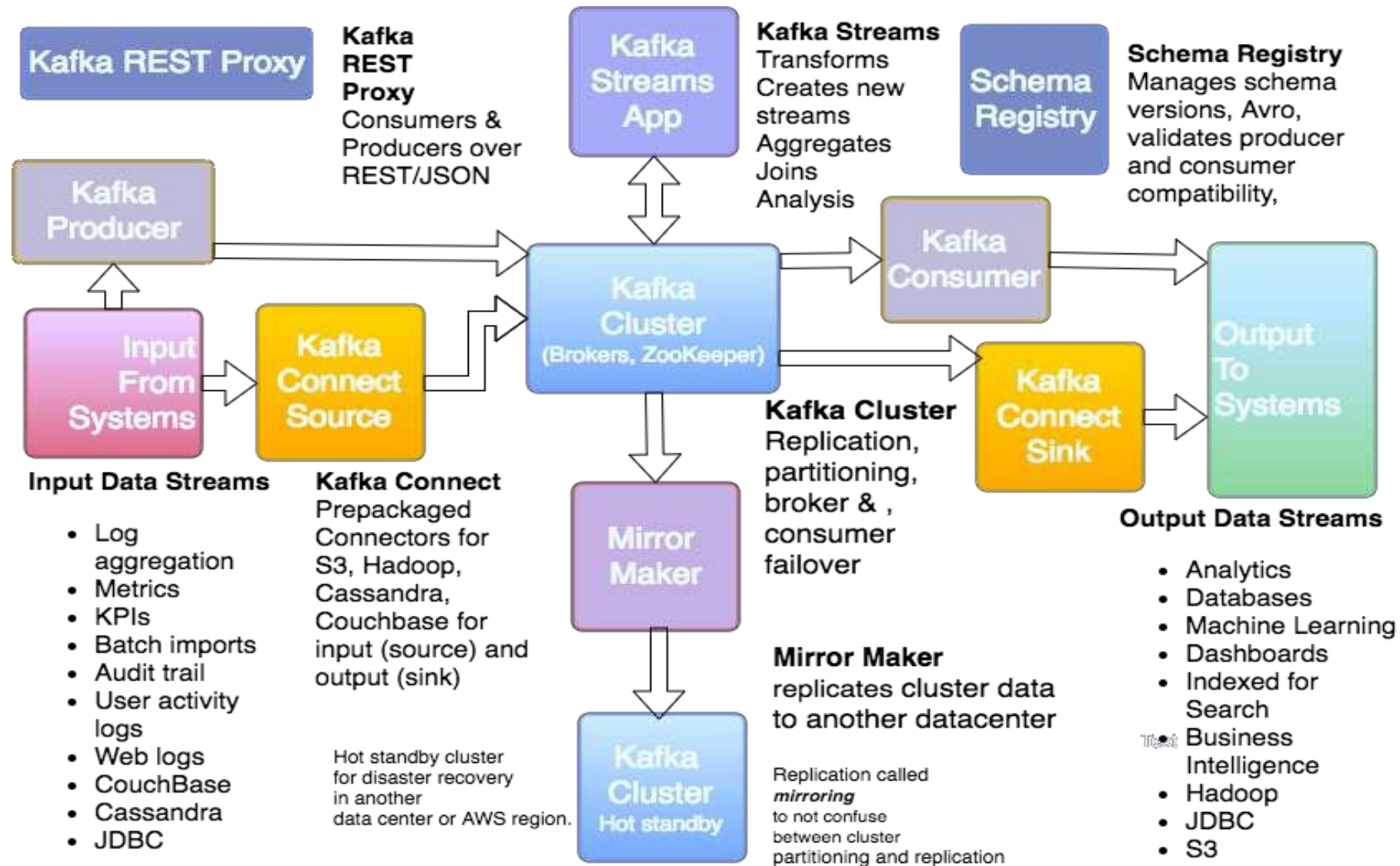
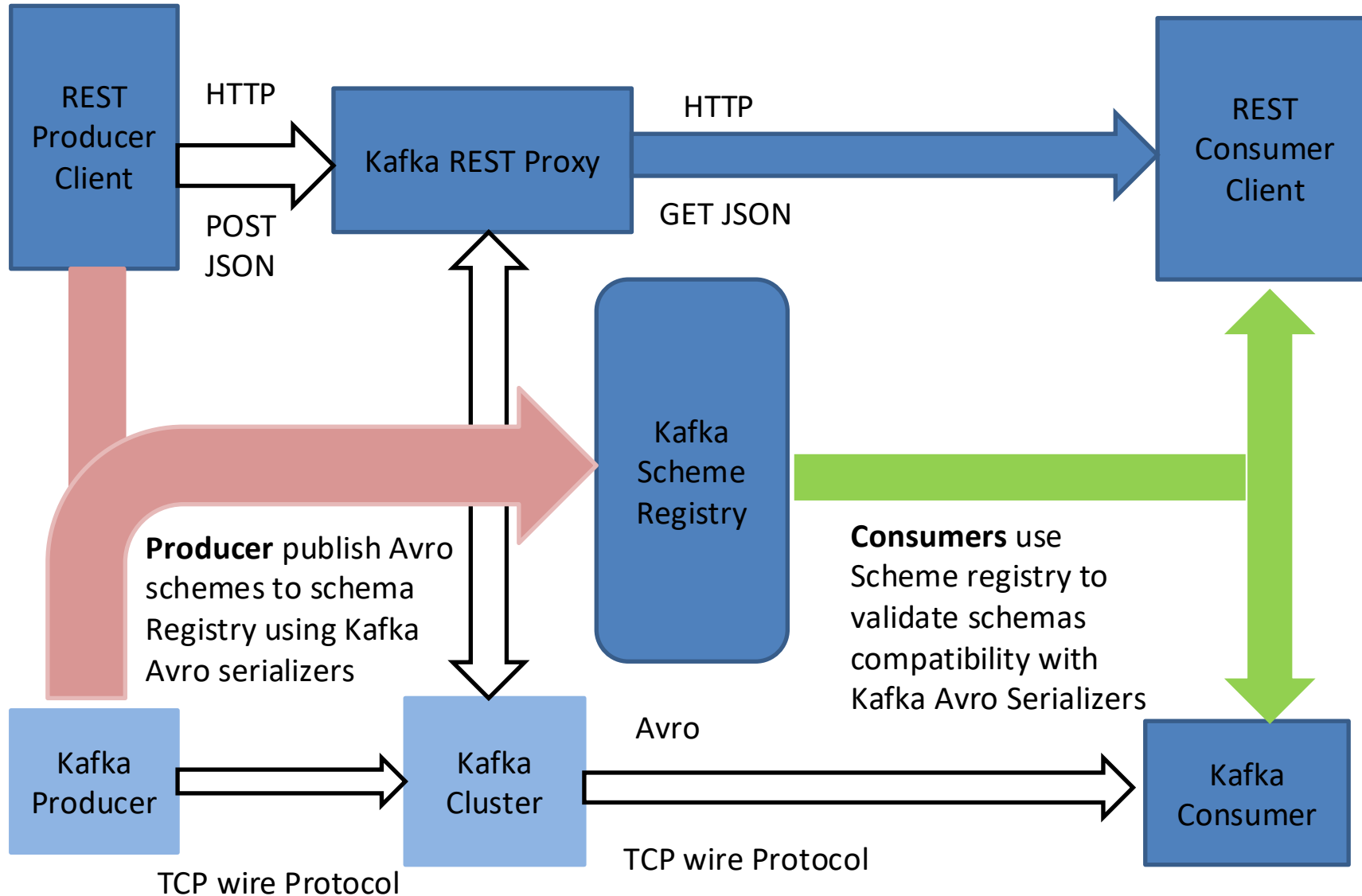


Kafka Ecosystem



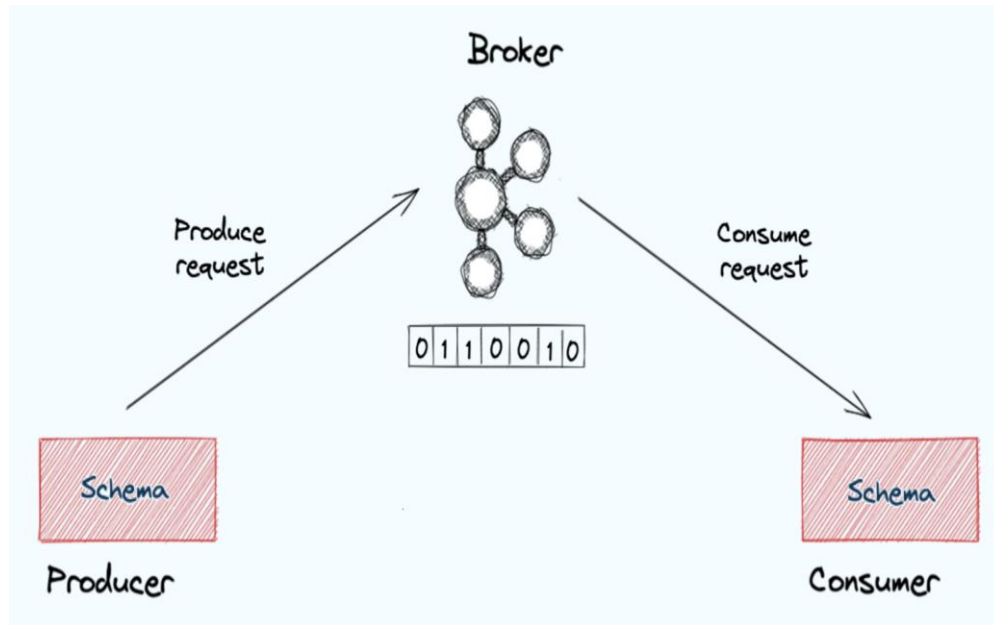
Kafka REST Proxy and Schema Registry

Tos



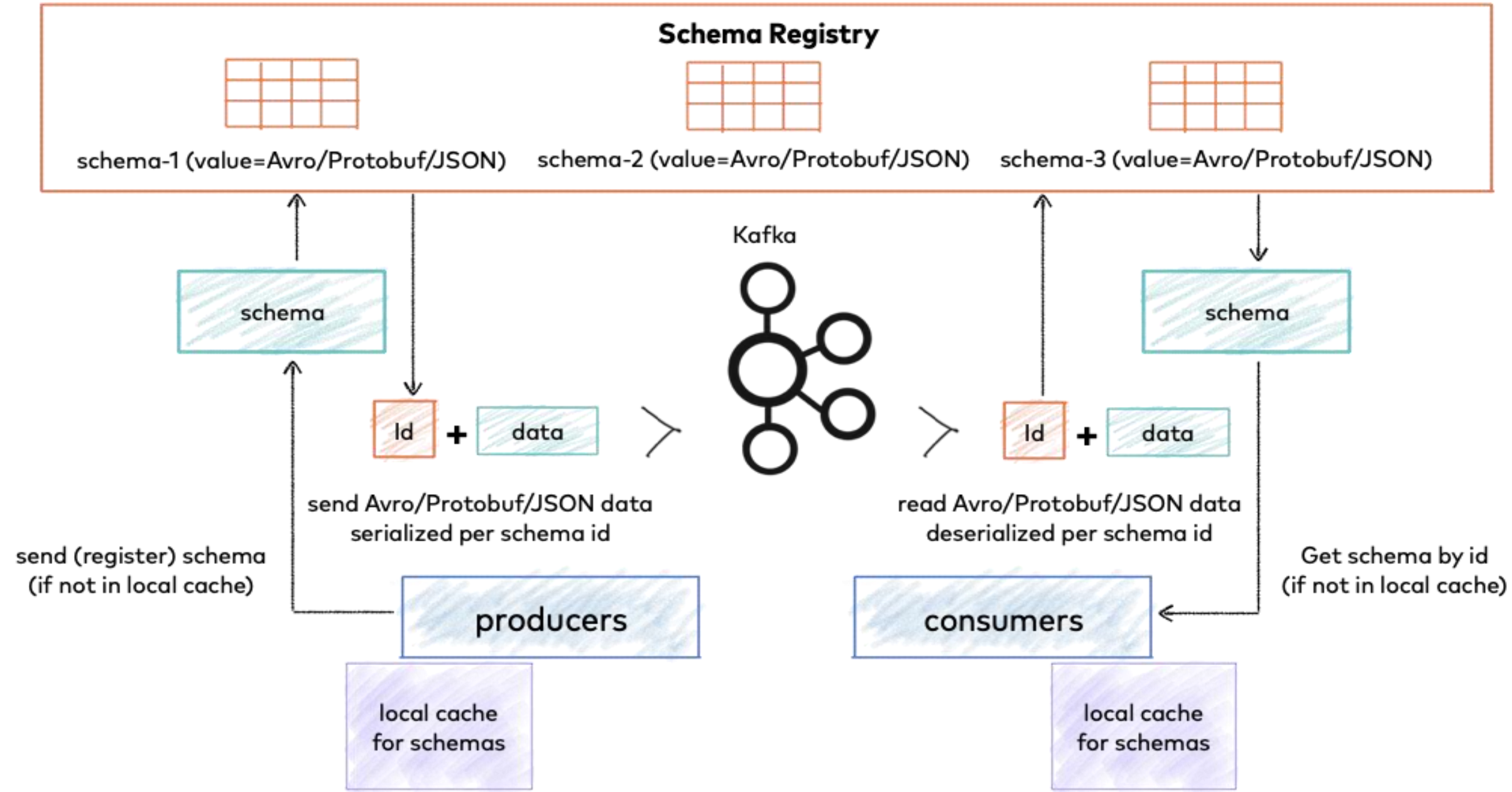
Schema Registry

Schema Is the Contract



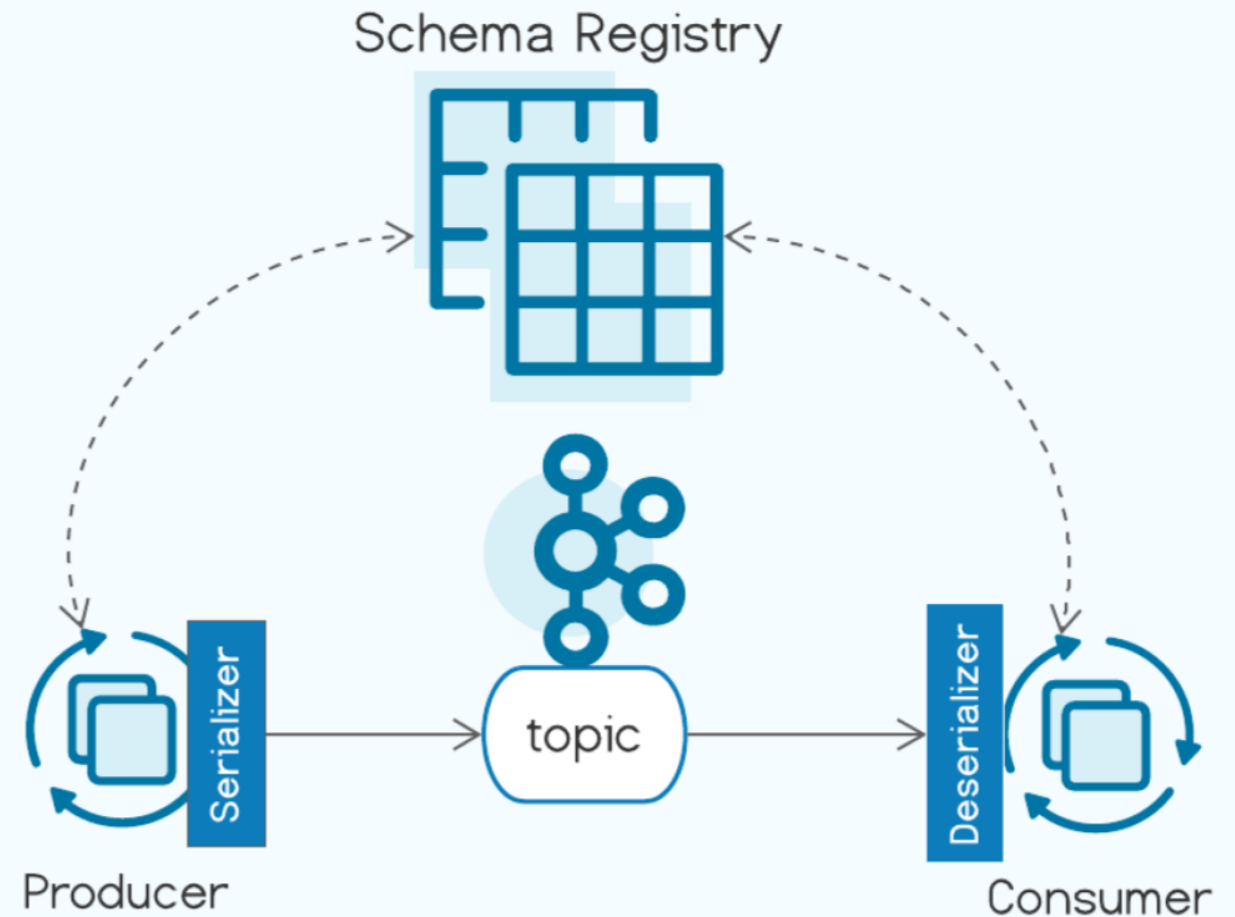
- A **schema** defines the structure and format of messages between a producer and a consumer.
- It acts as a **contract** between applications, ensuring both sides follow agreed rules.
- The schema specifies:
 - Field **names**
 - **Data types**
 - **Structure** of the message
- Violating the schema means breaking the contract — and can lead to failures or data issues.
- Enforcing schema compatibility helps maintain reliable communication.

Confluent Schema Registry



Confluent Schema Registry - Single Primary Architecture

- **Versioned** schema repository
- **Safe** schema evolution
- **Resilient** data pipelines
- **Enhanced** data integrity
- **Reduce** storage and computation
- **Discover** your data
- **Cost-efficient** ecosystem



Understanding the Schema Registry Workflow

The workflow of Schema Registry includes:

- Writing a schema file
- Adding schema files to a project
- Leveraging schema tools and plugins—Maven and Gradle
- Configuring Schema Registry plugins
- Generating the model objects from a schema
- Locating and exploring the generated files

Schema file - Protobuf

```
syntax = "proto3";  
package io.confluent.developer.proto;  
option java_outer_classname = "PurchaseProto";  
  
message Purchase {  
    string item = 1;  
    double total_cost = 2;  
    string customer_id = 3;  
}
```

- The first line declares the version of Protocol Buffers to use. This course uses version 3.
- The ***package*** declaration creates a name space for proto files to prevent name clashes. It ends up being the package name in the generated Java class unless you specify an ***option java_package*** in the proto file.
- The ***option java_outer_classname*** field describes the name of the generated java file
- Protocol Buffers define the ***message*** in the schema as an inner class in the generated file

Schema file - Avro

```
{
  "type": "record",
  "namespace": "io.confluent.developer.avro",
  "name": "Purchase",
  "fields": [
    {"name": "item", "type": "string"},
    {"name": "total_cost", "type": "double"},
    {"name": "customer_id", "type": "string"}
  ]
}
```

- The **namespace** field serves the same purpose as in the Protobuf file, preventing name collisions. The namespace also becomes the package name for the generated Java file.
- Fields are declared in a **JSON** array.

Schema Files in a Project

- The Avro schema files go in the **src/main/avro** directory and **Protobuf** files land in **src/main/proto**.

Once you've written your schema files you will want to generate the model/data objects from the schema files. Fortunately, there are plugins available, either Maven or Gradle, that can integrate into your local build.

```
schemaRegistry {  
    url = Confluent Cloud SR endpoint  
    credentials {  
        //characters up to the ':' in the basic.auth.user.info property  
        username = <username>  
        // password is everything after ':' in the basic.auth.user.info property  
        password = <password>  
    }  
    // Possible types are ["JSON", "PROTOBUF", "AVRO"]  
    register {  
        subject('avro-purchase', 'src/main/avro/purchase.avsc', 'AVRO')  
        subject('proto-purchase', 'src/main/proto/purchase.proto', 'PROTOBUF')  
    }  
}
```

Plugins to generate files

Generated Class files

▼ **src**

▼ **main**

▼ **avro**

- ≡ all_events.avsc
- ≡ customer_event.avsc
- ≡ customer_info.avsc
- ≡ page_view.avsc
- ≡ purchase.avsc

> **java**

▼ **proto**

- ≡ customer_event.proto
- ≡ page_view.proto
- ≡ purchase.proto

▼ build

- > classes
- > extracted-include-protos
- > extracted-protos
- > generated
- ▼ generated-main-avro-java
- ▼ io
- ▼ confluent
- ▼ developer
- ▼ avro
- Purchase.java
- ▼ generated-main-protobuf-java
- ▼ main
- ▼ java
- ▼ io
- ▼ confluent
- ▼ developer
- ▼ proto
- PurchaseProto.java

Confluent Schema Registry - Single Primary Architecture

ALL TOPICS >

transactions

Overview Messages Schema Configuration

Value Key

[Edit schema](#)

[Version history](#)

[Download](#)

Format: AVRO Version: 1

```
1  {
2    "fields": [
3      {
4        "name": "id",
5        "type": "string"
6      },
7      {
8        "name": "amount",
9        "type": "double"
10     }
11   ],
12   "name": "Payment",
13   "namespace": "io.confluent.examples.clients.basicavro",
14   "type": "record"
15 }
```

Take note of the version, schema ID, and the fields.

Confluent Schema Registry - Single Primary Architecture

etc/schema-registry/schema-registry.properties

```
# Specify the address the socket server listens on, e.g. listeners = PLAINTEXT://your.host.name:9092
listeners=http://0.0.0.0:8081

# The host name advertised in ZooKeeper. This must be specified if your running Schema Registry
# with multiple nodes.
host.name=192.168.50.1

# List of Kafka brokers to connect to, e.g. PLAINTEXT://hostname:9092,SSL://hostname2:9092
kafkastore.bootstrap.servers=PLAINTEXT://hostname:9092,SSL://hostname2:9092
```

Configuring Schema Registry API

Kafka applications using Avro data and Schema Registry need to specify at least two configuration parameters:

- Avro serializer or deserializer
- Properties to connect to Schema Registry

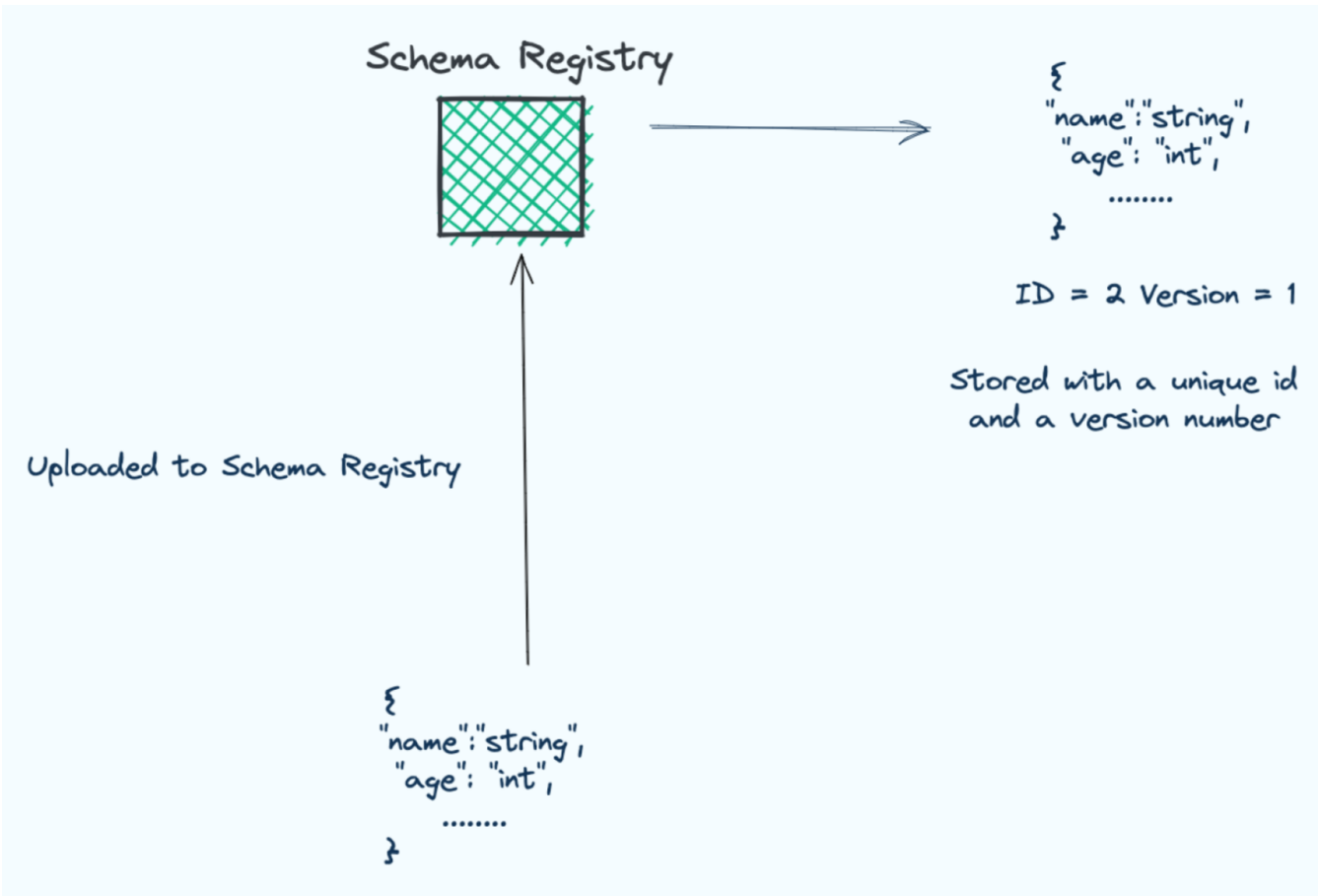
```
...
import io.confluent.kafka.serializers.KafkaAvroSerializer;
...
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, KafkaAvroSerializer.class);
...
KafkaProducer<String, Payment> producer = new KafkaProducer<String, Payment>(props);
final Payment payment = new Payment(orderId, 1000.00d);
final ProducerRecord<String, Payment> record = new ProducerRecord<String, Payment>(TOPIC, payment
.getId().toString(), payment);
producer.send(record);
...
```

```
...
import io.confluent.kafka.serializers.KafkaAvroDeserializer;
...
props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, KafkaAvroDeserializer.class);
props.put(KafkaAvroDeserializerConfig.SPECIFIC_AVRO_READER_CONFIG, true);
...
KafkaConsumer<String, Payment> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList(TOPIC));
while (true) {
    ConsumerRecords<String, Payment> records = consumer.poll(100);
    for (ConsumerRecord<String, Payment> record : records) {
        String key = record.key();
        Payment value = record.value();
    }
}
...
```

Effective schema management requires:

- Schema IDs
- Schema registration
- Schema versioning
- Viewing and retrieving schemas

Schema ID and Version



Schema management starts with registering schemas.

It also includes updating schemas and viewing or downloading them.

There are multiple ways to register a schema :

Confluent CLI, Schema Registry REST API, Confluent Cloud Console, or the Maven and Gradle plugins

Register an Avro Schema - REST API

```
jq '. | {schema: tojson}' purchase.avsc | curl -X POST -H \  
  "Content-Type: application/vnd.schemaregistry.v1+json" \  
  http://localhost:8081/subjects/my-kafka-value/versions -d \  
  @-
```

purchase.avsc

```
{  
  "type": "record",  
  "namespace": "tos.developer.avro",  
  "name": "Purchase",  
  "fields": [  
    {"name": "item", "type": "string"},  
    {"name": "total_cost", "type": "double"},  
    {"name": "customer_id", "type": "string"}  
  ]  
}
```

```
[root@kafka0 scripts]#  
[root@kafka0 scripts]# jq '. | {schema: tojson}' purchase.avsc | curl -X POST -H \  
> "Content-Type: application/vnd.schemaregistry.v1+json" \  
> http://localhost:8081/subjects/my-kafka-value/versions -d @-  
{"id":2}[root@kafka0 scripts]#
```

#dnf install jq

When you register a schema you need to provide the ***subject-name*** and the ***schema*** itself.

The ***subject-name*** is the ***name-space*** for the schema, almost like a key when you use a hash-map.

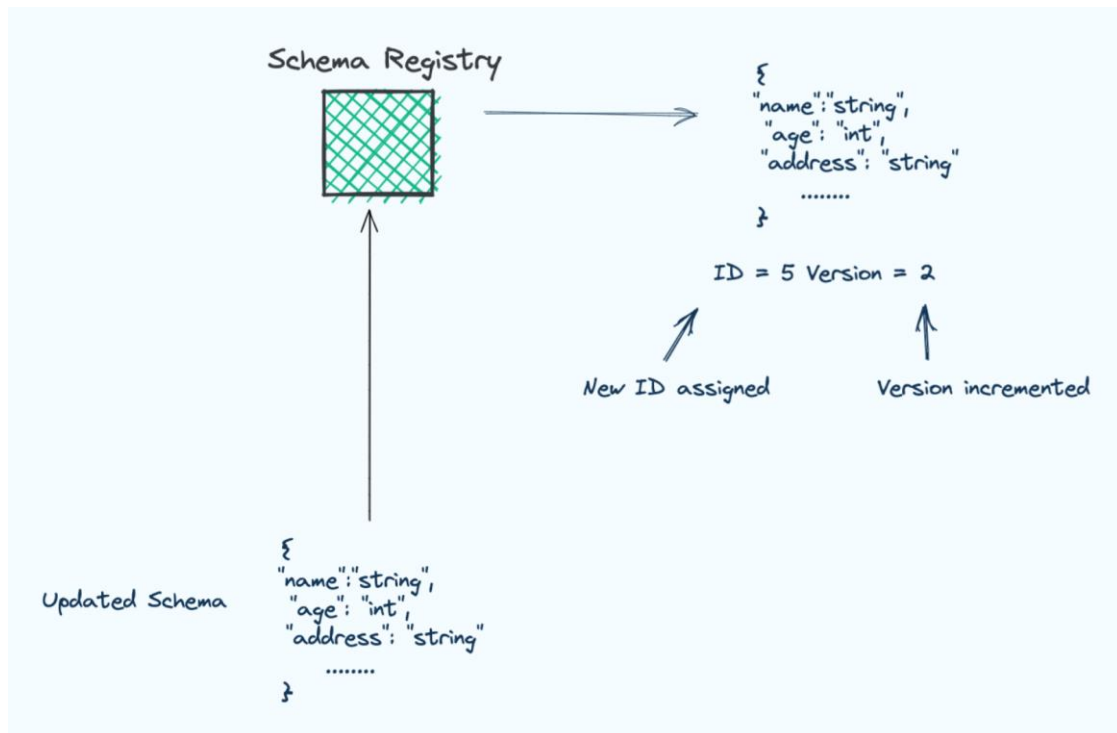
The standard naming convention is ***topic-name-key*** or ***topic-name-value***.

Once Schema Registry receives the ***schema***, it assigns it a unique ID number and a version number.

The first time you register a schema for a given ***subject-name***, it is assigned a version of 1.

Updating a Schema

Tos



To update previously registered schemas.
Use the same methods that was used to register the schema.

Provided you are making compatible changes (compatibility is covered in an upcoming module), Schema Registry will simply assign a new ID to the schema and a new version number.

The new ID is not guaranteed to be in sequential order.

But the version number will always be incremented by one, hence it will be in sequential order

```
curl -s "http://localhost:8081/schemas/ids/2" | jq
```

```
[root@kafka0 scripts]# curl -s "http://localhost:8081/schemas/ids/2" | jq
{
  "schema": "{\\\"type\\\":\\\"record\\\",\\\"name\\\":\\\"Purchase\\\",\\\"namespace\\\":\\\"tos.developer.avro\\\",\\\"fields\\\":[{\\\"name\\\":\\\"item\\\",\\\"type\\\":\\\"string\\\"},{\\\"name\\\":\\\"total_cost\\\",\\\"type\\\":\\\"double\\\"},{\\\"name\\\":\\\"customer_id\\\",\\\"type\\\":\\\"string\\\"}]}\""}
[root@kafka0 scripts]#
```

Lab : Schema Registry - Manage Schemas for Topics